Floating Dictionary — complete source code

Complete text source and build files for the Android app. The launcher icon uses the latest uploaded square dictionary illustration; scaled PNG assets are under `app/src/main/res/mipmap-*`, with the original image under `branding/`. These binary assets are included in the GitHub ZIP. The user-visible app version is 1.0; the internal release designation remains 1.5. The Gradle wrapper JAR is also included in the ZIP.

settings.gradle

Plain Text

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = 'Floating-Dictionary'
include ':app'
```

build.gradle

Plain Text

```
plugins {
    id 'com.android.application' version '8.6.1' apply false
    id 'org.jetbrains.kotlin.android' version '2.0.21' apply false
}
```

gradle.properties

Plain Text

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
org.gradle.parallel=true
```

gradle/wrapper/gradle-wrapper.properties

Plain Text

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.7-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

gradlew

Plain Text

```
#!/bin/sh

#
# Copyright © 2015-2021 the original authors.
#
# Licensed under the Apache License, Version 2.0 (the "License" );
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#

##############################################################################
#
#   Gradle start up script for POSIX generated by Gradle.
```

```
#
#   Important for running:
#
#   (1 ) You need a POSIX-compliant shell to run this script. If your /bin/sh i
#       noncompliant, but you have some other compliant shell such as ksh or
#       bash, then to run this script, type that shell name before the whole
#       command line, like:
#
#           ksh Gradle
#
#       Busybox and similar reduced shells will NOT work, because this script
#       requires all of these POSIX shell features:
#         * functions;
#         * expansions «$var», «${var}», «${var:-default}», «${var+SET}»,
#           «${var#prefix}», «${var%suffix}», and «$( cmd )»;
#         * compound commands having a testable exit status, especially «case»;
#         * various built-in commands including «command», «set», and «ulimit».
#
#   Important for patching:
#
#   (2) This script targets any POSIX shell, so it avoids extensions provided
#       by Bash, Ksh, etc; in particular arrays are avoided.
#
#       The "traditional" practice of packing multiple parameters into a
#       space-separated string is a well documented source of bugs and security
#       problems, so this is (mostly) avoided, by progressively accumulating
#       options in "$@", and eventually passing that to Java.
#
#       Where the inherited environment variables (DEFAULT_JVM_OPTS, JAVA_OPTS,
#       and GRADLE_OPTS) rely on word-splitting, this is performed explicitly;
#       see the in-line comments for details.
#
#       There are tweaks for specific operating systems such as AIX, CygWin,
#       Darwin, MinGW, and NonStop.
#
#   (3) This script is generated from the Groovy template
#       https://github.com/gradle/gradle/blob/HEAD/subprojects/plugins/src/main
#       within the Gradle project.
#
#       You can find Gradle at https://github.com/gradle/gradle/.
#
##############################################################################

# Attempt to set APP_HOME

# Resolve links: $0 may be a link
app_path=$0
```

```
# Need this for daisy-chained symlinks.
while
    APP_HOME=${app_path%"${app_path##*/}"}  # leaves a trailing /; empty if no
    [ -h "$app_path" ]
do
    ls=$( ls -ld "$app_path"  )
    link=${ls#*' -> '}
    case $link in             #(
      /*)   app_path=$link ;; #(
      *)    app_path=$APP_HOME$link ;;
    esac
done

# This is normally unused
# shellcheck disable=SC2034
APP_BASE_NAME=${0##*/}
# Discard cd standard output in case $CDPATH is set (https://github.com/gradle/
APP_HOME=$( cd "${APP_HOME:-./}" > /dev/null && pwd -P ) || exit

# Use the maximum available, or set MAX_FD != -1 to use that value.
MAX_FD=maximum

warn () {
    echo "$*"
} >&2

die () {
    echo
    echo "$*"
    echo
    exit 1
} >&2

# OS specific support (must be 'true' or 'false').
cygwin=false
msys=false
darwin=false
nonstop=false
case "$( uname )" in                #(
  CYGWIN* )         cygwin=true  ;; #(
  Darwin* )         darwin=true  ;; #(
  MSYS* | MINGW* )  msys=true    ;; #(
  NONSTOP* )        nonstop=true ;;
esac

CLASSPATH=$APP_HOME/gradle/wrapper/gradle-wrapper.jar
```

```
# Determine the Java command to use to start the JVM.
if [ -n "$JAVA_HOME" ] ; then
    if [ -x "$JAVA_HOME/jre/sh/java" ] ; then
        # IBM's JDK on AIX uses strange locations for the executables
        JAVACMD=$JAVA_HOME/jre/sh/java
    else
        JAVACMD=$JAVA_HOME/bin/java
    fi
    if [ ! -x "$JAVACMD" ] ; then
        die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
else
    JAVACMD=java
    if ! command -v java >/dev/null 2>&1
    then
        die "ERROR: JAVA_HOME is not set and no 'java' command could be found i

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
fi

# Increase the maximum file descriptors if we can.
if ! "$cygwin" && ! "$darwin" && ! "$nonstop" ; then
    case $MAX_FD in #(
      max*)
        # In POSIX sh, ulimit -H is undefined. That's why the result is checked
        # shellcheck disable=SC2039,SC3045
        MAX_FD=$( ulimit -H -n ) ||
            warn "Could not query maximum file descriptor limit"
    esac
    case $MAX_FD in  #(
      '' | soft) :;; #(
      *)
        # In POSIX sh, ulimit -n is undefined. That's why the result is checked
        # shellcheck disable=SC2039,SC3045
        ulimit -n "$MAX_FD" ||
            warn "Could not set maximum file descriptor limit to $MAX_FD"
    esac
fi

# Collect all arguments for the java command, stacking in reverse order:
#   * args from the command line
#   * the main class name
#   * -classpath
```

```
#   * -D...appname settings
#   * --module-path (only if needed)
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and GRADLE_OPTS environment variables.

# For Cygwin or MSYS, switch paths to Windows format before running java
if "$cygwin" || "$msys" ; then
    APP_HOME=$( cygpath --path --mixed "$APP_HOME" )
    CLASSPATH=$( cygpath --path --mixed "$CLASSPATH" )

    JAVACMD=$( cygpath --unix "$JAVACMD" )

    # Now convert the arguments - kludge to limit ourselves to /bin/sh
    for arg do
        if
            case $arg in                                #(
              -*)   false ;;                            # don't mess with optio
              /?*)  t=${arg#/} t=/${t%%/*}              # looks like a POSIX fi
                    [ -e "$t" ] ;;                      #(
              *)    false ;;
            esac
        then
            arg=$( cygpath --path --ignore --mixed "$arg" )
        fi
        # Roll the args list around exactly as many times as the number of
        # args, so each arg winds up back in the position where it started, but
        # possibly modified.
        #
        # NB: a `for` loop captures its iteration list before it begins, so
        # changing the positional parameters here affects neither the number of
        # iterations, nor the values presented in `arg`.
        shift                   # remove old arg
        set -- "$@" "$arg"      # push replacement arg
    done
fi


# Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to p
DEFAULT_JVM_OPTS='"-Xmx64m" "-Xms64m"'

# Collect all arguments for the java command:
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, JAVA_OPTS, and optsEnvironmentVar are not al
#     and any embedded shellness will be escaped.
#   * For example: A user cannot expect ${Hostname} to be expanded, as it is an
#     treated as '${Hostname}' itself on the command line.

set -- \
        "-Dorg.gradle.appname=$APP_BASE_NAME" \
        -classpath "$CLASSPATH" \
```

```
        org.gradle.wrapper.GradleWrapperMain \
        "$@"

# Stop when "xargs" is not available.
if ! command -v xargs >/dev/null 2>&1
then
    die "xargs is not available"
fi

# Use "xargs" to parse quoted args.
#
# With -n1 it outputs one arg per line, with the quotes and backslashes removed
#
# In Bash we could simply go:
#
#   readarray ARGS < <( xargs -n1 <<<"$var" ) &&
#   set -- "${ARGS[@]}" "$@"
#
# but POSIX shell has neither arrays nor command substitution, so instead we
# post-process each arg (as a line of input to sed) to backslash-escape any
# character that might be a shell metacharacter, then use eval to reverse
# that process (while maintaining the separation between arguments), and wrap
# the whole thing up as a single "set" statement.
#
# This will of course break if any of these variables contains a newline or
# an unmatched quote.
#

eval "set -- $(
        printf '%s\n' "$DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS" |
        xargs -n1 |
        sed ' s~[^-[:alnum:]+,./:=@_]~\\&~g; ' |
        tr '\n' ' '
    )" '"$@"'

exec "$JAVACMD" "$@"
```

gradlew.bat

Plain Text

```
@rem
@rem Copyright 2015 the original author or authors.
@rem
@rem Licensed under the Apache License, Version 2.0 (the "License");
@rem you may not use this file except in compliance with the License.
```

```
@rem You may obtain a copy of the License at
@rem
@rem      https://www.apache.org/licenses/LICENSE-2.0
@rem
@rem Unless required by applicable law or agreed to in writing, software
@rem distributed under the License is distributed on an "AS IS" BASIS,
@rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
@rem See the License for the specific language governing permissions and
@rem limitations under the License.
@rem

@if "%DEBUG%"=="" @echo off
@rem ##########################################################################
@rem
@rem  Gradle startup script for Windows
@rem
@rem ##########################################################################

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

set DIRNAME=%~dp0
if "%DIRNAME%"=="" set DIRNAME=.
@rem This is normally unused
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Resolve any "." and ".." in APP_HOME to make it shorter.
for %%i in ("%APP_HOME%" ) do set APP_HOME=%%~fi

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS t
set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if %ERRORLEVEL% equ 0 goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your P
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail
```

```
:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:execute
@rem Setup the command line

set CLASSPATH=%APP_HOME%\gradle\wrapper\gradle-wrapper.jar


@rem Execute Gradle
"%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname

:end
@rem End local scope for the variables with windows NT shell
if %ERRORLEVEL% equ 0 goto mainEnd

:fail
rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code inste
rem the _cmd.exe /c_ return code!
set EXIT_CODE=%ERRORLEVEL%
if %EXIT_CODE% equ 0 set EXIT_CODE=1
if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
exit /b %EXIT_CODE%

:mainEnd
if "%OS%"=="Windows_NT" endlocal

:omega
```

.gitignore

Plain Text

```
.gradle/
/build/
```

```
/app/build/
/local.properties
*.iml
.idea/
.DS_Store
```

app/build.gradle

Plain Text

```
plugins {
    id 'com.android.application'
    id 'org.jetbrains.kotlin.android'
}

android {
    namespace 'com.example.floatingdictionary'
    compileSdk 35

    defaultConfig {
        applicationId 'com.example.floatingdictionary'
        minSdk 29
        targetSdk 29
        // Internal release designation remains 1.5; Android shows versionName
        versionCode 7
        versionName '1.0'
    }

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }

    kotlinOptions {
        jvmTarget = '1.8'
    }
}
```

app/src/main/AndroidManifest.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
```

```
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <service
            android:name=".FloatingService"
            android:exported="false" />
    </application>
</manifest>
```

app/src/main/java/com/example/floatingdictionary/MainActivity.kt

Kotlin

```
package com.example.floatingdictionary

import android.app.Activity
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.Bundle
import android.provider.Settings
import android.widget.Button
import android.widget.TextView

class MainActivity : Activity( ) {
```

```
    private var startAfterPermission = false
    private lateinit var statusText: TextView
    private lateinit var actionButton: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        statusText = findViewById(R.id.permissionStatus)
        actionButton = findViewById(R.id.actionButton)
        actionButton.setOnClickListener {
            if (Settings.canDrawOverlays(this)) {
                startFloatingService()
            } else {
                startAfterPermission = true
                val permissionIntent = Intent(
                    Settings.ACTION_MANAGE_OVERLAY_PERMISSION,
                    Uri.parse("package:$packageName")
                )
                startActivity(permissionIntent)
            }
        }
        updatePermissionStatus()
    }

    override fun onResume() {
        super.onResume()
        updatePermissionStatus()
        if (startAfterPermission && Settings.canDrawOverlays(this)) {
            startAfterPermission = false
            startFloatingService()
        }
    }

    private fun updatePermissionStatus() {
        val hasPermission = Settings.canDrawOverlays(this)
        statusText.text = if (hasPermission) {
            "Overlay permission is enabled. Start the floating bubble whenever
        } else {
            "Allow display over other apps so the dictionary bubble can appear
        }
        actionButton.text = if (hasPermission) "Start floating bubble" else "Gr
    }

    private fun startFloatingService() {
        val serviceIntent = Intent(this, FloatingService::class.java)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(serviceIntent)
```

```
        } else {
            startService(serviceIntent)
        }
        statusText.text = "Floating Dictionary is running. Tap its bubble to se
    }
}
```

app/src/main/java/com/example/floatingdictionary/FloatingService.kt

Kotlin

```
package com.example.floatingdictionary

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.app.Service
import android.content.Context
import android.content.Intent
import android.graphics.PixelFormat
import android.os.Build
import android.os.Handler
import android.os.IBinder
import android.os.Looper
import android.provider.Settings
import android.text.Html
import android.view.Gravity
import android.view.MotionEvent
import android.view.View
import android.view.ViewConfiguration
import android.view.WindowManager
import android.view.inputmethod.InputMethodManager
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import org.json.JSONArray
import org.json.JSONObject
import java.io.BufferedReader
import java.io.InputStreamReader
import java.net.HttpURLConnection
import java.net.URL
import java.net.URLEncoder
import java.util.concurrent.ExecutorService
import java.util.concurrent.Executors
```

```kotlin
import kotlin.math.abs

class FloatingService : Service() {

    companion object {
        private const val CHANNEL_ID = "floating_dictionary_service"
        private const val NOTIFICATION_ID = 1001
    }

    private lateinit var windowManager: WindowManager
    private var overlayView: View? = null
    private var overlayParams: WindowManager.LayoutParams? = null
    private var panelOpen = false
    private val mainHandler = Handler(Looper.getMainLooper())
    private val executor: ExecutorService = Executors.newSingleThreadExecutor()

    override fun onCreate() {
        super.onCreate()
        windowManager = getSystemService(Context.WINDOW_SERVICE) as WindowManag
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int
        createNotificationChannel()
        startForeground(NOTIFICATION_ID, buildNotification())

        if (Settings.canDrawOverlays(this) && overlayView == null) {
            showOverlay()
        } else if (!Settings.canDrawOverlays(this)) {
            stopSelf()
        }
        return START_STICKY
    }

    private fun showOverlay() {
        val root = View.inflate(this, R.layout.floating_layout, null)
        val bubble = root.findViewById<TextView>(R.id.floatingBubble)
        val panel = root.findViewById<View>(R.id.searchPanel)
        val wordInput = root.findViewById<EditText>(R.id.wordInput)
        val searchButton = root.findViewById<Button>(R.id.searchButton)
        val closeButton = root.findViewById<Button>(R.id.closeButton)
        val stopBubbleButton = root.findViewById<Button>(R.id.stopBubbleButton)
        val definitionText = root.findViewById<TextView>(R.id.definitionText)

        val overlayType = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY
        } else {
            @Suppress("DEPRECATION")
            WindowManager.LayoutParams.TYPE_PHONE
```

```
        }
        val initialFlags = WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
            WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL
        val params = WindowManager.LayoutParams(
            WindowManager.LayoutParams.WRAP_CONTENT,
            WindowManager.LayoutParams.WRAP_CONTENT,
            overlayType,
            initialFlags,
            PixelFormat.TRANSLUCENT
        ).apply {
            gravity = Gravity.TOP or Gravity.START
            x = dp(16)
            y = dp(120)
        }

        overlayView = root
        overlayParams = params
        try {
            windowManager.addView(root, params)
        } catch (exception: Exception) {
            overlayView = null
            overlayParams = null
            stopSelf()
            return
        }

        val touchSlop = ViewConfiguration.get(this).scaledTouchSlop
        bubble.setOnTouchListener(object : View.OnTouchListener {
            private var downRawX = 0f
            private var downRawY = 0f
            private var startX = 0
            private var startY = 0
            private var moved = false

            override fun onTouch(view: View, event: MotionEvent): Boolean {
                val currentParams = overlayParams ?: return false
                when (event.action) {
                    MotionEvent.ACTION_DOWN -> {
                        downRawX = event.rawX
                        downRawY = event.rawY
                        startX = currentParams.x
                        startY = currentParams.y
                        moved = false
                        return true
                    }
                    MotionEvent.ACTION_MOVE -> {
                        val dx = event.rawX - downRawX
                        val dy = event.rawY - downRawY
```

```
                    if (abs(dx) > touchSlop || abs(dy) > touchSlop) moved =
                    if (moved) {
                        currentParams.x = startX + dx.toInt()
                        currentParams.y = startY + dy.toInt()
                        try {
                            windowManager.updateViewLayout(root, currentPar
                        } catch (_: Exception) {
                            // The overlay may have been removed while the
                        }
                    }
                    return true
                }
                MotionEvent.ACTION_UP -> {
                    if (!moved) togglePanel(panel, wordInput)
                    return true
                }
                MotionEvent.ACTION_CANCEL -> return true
            }
            return true
        }
    })

    searchButton.setOnClickListener {
        val word = wordInput.text.toString().trim()
        if (word.isEmpty()) {
            definitionText.text = "Type a word to look it up."
        } else {
            lookupWord(word, definitionText)
        }
    }
    closeButton.setOnClickListener { closePanel(panel, wordInput) }
    stopBubbleButton.setOnClickListener {
        closePanel(panel, wordInput)
        stopSelf()
    }
}

private fun togglePanel(panel: View, wordInput: EditText) {
    if (panelOpen) {
        closePanel(panel, wordInput)
        return
    }
    val root = overlayView ?: return
    val params = overlayParams ?: return
    panel.visibility = View.VISIBLE
    panelOpen = true
    params.flags = params.flags and WindowManager.LayoutParams.FLAG_NOT_FOC
    try {
```

```
            windowManager.updateViewLayout(root, params)
        } catch (_: Exception) {
            return
        }
        wordInput.requestFocus()
        wordInput.post {
            val inputMethodManager = getSystemService(Context.INPUT_METHOD_SERV
            inputMethodManager.showSoftInput(wordInput, InputMethodManager.SHOW_
        }
    }

    private fun closePanel(panel: View, wordInput: EditText) {
        val root = overlayView ?: return
        val params = overlayParams ?: return
        val inputMethodManager = getSystemService(Context.INPUT_METHOD_SERVICE)
        inputMethodManager.hideSoftInputFromWindow(wordInput.windowToken, 0)
        panel.visibility = View.GONE
        wordInput.clearFocus()
        panelOpen = false
        params.flags = params.flags or WindowManager.LayoutParams.FLAG_NOT_FOCUS
        try {
            windowManager.updateViewLayout(root, params)
        } catch (_: Exception) {
            // Service may be shutting down.
        }
    }

    private fun lookupWord(word: String, output: TextView) {
        output.text = "Searching…"
        executor.execute {
            val result = try {
                val encodedWord = URLEncoder.encode(word, "UTF-8").replace("+",
                val connection = URL("https://en.wiktionary.org/api/rest_v1/pag
                    .openConnection() as HttpURLConnection
                try {
                    connection.requestMethod = "GET"
                    connection.connectTimeout = 15_000
                    connection.readTimeout = 20_000
                    connection.setRequestProperty("Accept", "application/json")
                    connection.setRequestProperty(
                        "User-Agent",
                        "FloatingDictionary/1.0 (Android; English Wiktionary lo
                    )
                    val status = connection.responseCode
                    val stream = if (status in 200..299) connection.inputStream
                    val response = if (stream != null) {
                        BufferedReader(InputStreamReader(stream, Charsets.UTF_8
                    } else {
```

```
                        ""
                    }
                    if (status in 200..299) {
                        formatDefinition(response, word)
                    } else if (status == HttpURLConnection.HTTP_NOT_FOUND) {
                        "No English Wiktionary entry was found for ‘$word’."
                    } else {
                        "Wiktionary returned HTTP $status. Please try again."
                    }
                } finally {
                    connection.disconnect()
                }
            } catch (exception: Exception) {
                val reason = if (exception is java.net.SocketTimeoutException)
                    "The request timed out. Check your connection and try again
                } else {
                    exception.localizedMessage ?: "check your internet connecti
                }
                "Could not reach English Wiktionary: $reason"
            }
            mainHandler.post {
                if (overlayView != null && output.isAttachedToWindow) output.te
            }
        }
    }

    private fun formatDefinition(response: String, requestedWord: String): Stri
        val responseObject = JSONObject(response)
        val entries = responseObject.optJSONArray("en") ?: JSONArray()
        val output = StringBuilder()
        val synonyms = LinkedHashSet<String>()
        var definitionsAdded = 0

        for (entryIndex in 0 until entries.length()) {
            val entry = entries.optJSONObject(entryIndex) ?: continue
            if (output.isEmpty()) {
                output.append("English Wiktionary: ").append(requestedWord).app
            }
            val partOfSpeech = entry.optString("partOfSpeech", "")
            if (partOfSpeech.isNotBlank()) output.append(partOfSpeech).append('
            val definitions = entry.optJSONArray("definitions") ?: JSONArray()
            for (definitionIndex in 0 until definitions.length()) {
                val definition = definitions.optJSONObject(definitionIndex) ?:
                val html = definition.optString("definition", "")
                val text = Html.fromHtml(html, Html.FROM_HTML_MODE_LEGACY)
                    .toString()
                    .replace(Regex("\\s+"), " ")
                    .trim()
```

```
                if (text.isNotBlank() && definitionsAdded < 6) {
                    output.append("• ").append(text).append('\n')
                    definitionsAdded++
                }
                addJsonStrings(synonyms, definition.optJSONArray("synonyms"))
            }
            addJsonStrings(synonyms, entry.optJSONArray("synonyms"))
            output.append('\n')
        }

        if (definitionsAdded == 0) {
            return "Wiktionary did not return an English definition for ‘$reque
        }
        if (synonyms.isNotEmpty()) {
            output.append("Synonyms: ").append(synonyms.take(12).joinToString("
        } else {
            output.append("Synonyms: see the Wiktionary entry below")
        }
        val pageTitle = URLEncoder.encode(requestedWord.replace(' ', '_'), "UTF
            .replace("+", "%20")
        output.append("\n\nSource: English Wiktionary (CC BY-SA)\n")
            .append("https://en.wiktionary.org/wiki/" )
            .append(pageTitle)
        return output.toString().trim()
    }

    private fun addJsonStrings(destination: MutableSet<String>, values: JSONArr
        if (values == null) return
        for (index in 0 until values.length()) {
            val value = values.optString(index).trim()
            if (value.isNotEmpty()) destination.add(value)
        }
    }

    private fun dp(value: Int): Int = (value * resources.displayMetrics.density

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Floating Dictionary",
                NotificationManager.IMPORTANCE_LOW
            ).apply {
                description = "Keeps the floating dictionary bubble available"
            }
            val manager = getSystemService(Context.NOTIFICATION_SERVICE) as Not
            manager.createNotificationChannel(channel)
        }
```

```
    }

    private fun buildNotification(): Notification {
        val activityIntent = Intent(this, MainActivity::class.java)
        val pendingIntent = PendingIntent.getActivity(
            this,
            0,
            activityIntent,
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
        )
        return Notification.Builder(this, CHANNEL_ID)
            .setContentTitle("Floating Dictionary is running")
            .setContentText("Tap the floating bubble to search for a word")
            .setSmallIcon(android.R.drawable.ic_menu_search)
            .setContentIntent(pendingIntent)
            .setOngoing(true)
            .build()
    }

    override fun onDestroy() {
        val view = overlayView
        if (view != null) {
            try {
                windowManager.removeViewImmediate(view)
            } catch (_: Exception) {
                // It may already have been removed by the system.
            }
        }
        overlayView = null
        overlayParams = null
        executor.shutdownNow()
        stopForeground(true)
        super.onDestroy()
    }

    override fun onBind(intent: Intent?): IBinder? = null
}
```

app/src/main/res/layout/activity_main.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
    android:gravity="center_vertical"
    android:orientation="vertical"
    android:padding="24dp">

    <ImageView
        android:layout_width="136dp"
        android:layout_height="136dp"
        android:layout_gravity="center_horizontal"
        android:layout_marginBottom="16dp"
        android:contentDescription="@string/app_name"
        android:scaleType="centerInside"
        android:src="@mipmap/ic_launcher" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/app_name"
        android:textColor="#202124"
        android:textSize="26sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/permissionStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:layout_marginBottom="24dp"
        android:text="Checking overlay permission…"
        android:textColor="#444444"
        android:textSize="16sp" />

    <Button
        android:id="@+id/actionButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Grant permission and start"
        android:textAllCaps="false" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="When the bubble appears, drag it anywhere or tap it to op
        android:textColor="#666666"
        android:textSize="14sp" />
</LinearLayout>
```

app/src/main/res/layout/floating_layout.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/floatingRoot"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="vertical">

    <TextView
        android:id="@+id/floatingBubble"
        android:layout_width="56dp"
        android:layout_height="56dp"
        android:background="@drawable/bubble_background"
        android:contentDescription="Floating Dictionary: tap to search, drag to
        android:elevation="6dp"
        android:gravity="center"
        android:text="D"
        android:textColor="#FFFFFF"
        android:textSize="24sp"
        android:textStyle="bold" />

    <LinearLayout
        android:id="@+id/searchPanel"
        android:layout_width="300dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:background="@drawable/panel_background"
        android:elevation="10dp"
        android:orientation="vertical"
        android:padding="14dp"
        android:visibility="gone">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Dictionary lookup"
                android:textColor="#202124"
```

```
            android:textSize="18sp"
            android:textStyle="bold" />

        <Button
            android:id="@+id/stopBubbleButton"
            android:layout_width="76dp"
            android:layout_height="44dp"
            android:contentDescription="Turn off the floating bubble"
            android:minWidth="0dp"
            android:minHeight="0dp"
            android:text="Turn off"
            android:textAllCaps="false"
            android:textSize="12sp" />

        <Button
            android:id="@+id/closeButton"
            android:layout_width="44dp"
            android:layout_height="44dp"
            android:contentDescription="Close dictionary panel"
            android:minWidth="0dp"
            android:minHeight="0dp"
            android:text="×"
            android:textAllCaps="false"
            android:textSize="22sp" />
    </LinearLayout>

    <EditText
        android:id="@+id/wordInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:hint="Type an English word"
        android:imeOptions="actionSearch"
        android:inputType="text"
        android:maxLines="1"
        android:singleLine="true" />

    <Button
        android:id="@+id/searchButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Search"
        android:textAllCaps="false" />

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="190dp"
```

```
            android:layout_marginTop="8dp"
            android:fillViewport="true">

            <TextView
                android:id="@+id/definitionText"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:autoLink="web"
                android:linksClickable="true"
                android:text="Definitions and synonyms will appear here."
                android:textColor="#333333"
                android:textSize="14sp" />
        </ScrollView>
    </LinearLayout>
</LinearLayout>
```

app/src/main/res/drawable/bubble_background.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="#6750A4" />
    <stroke android:width="2dp" android:color="#FFFFFF" />
</shape>
```

app/src/main/res/drawable/panel_background.xml

XML

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape
    <solid android:color="#FFFFFF" />
    <corners android:radius="14dp" />
    <stroke android:width="1dp" android:color="#DDDDDD" />
</shape>
```

app/src/main/res/values/styles.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="AppTheme" parent="@android:style/Theme.Material.Light.NoAction
        <item name="android:fontFamily">sans</item>
        <item name="android:colorAccent">#6750A4</item>
        <item name="android:statusBarColor">#FFFFFF</item>
        <item name="android:navigationBarColor">#FFFFFF</item>
        <item name="android:windowLightStatusBar">true</item>
    </style>
</resources>
```

app/src/main/res/values/colors.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="launcher_background">#F5FAFD</color>
</resources>
```

app/src/main/res/values/strings.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">Floating Dictionary</string>
</resources>
```

app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/launcher_background" />
    <foreground android:drawable="@mipmap/ic_launcher_foreground" />
</adaptive-icon>
```

app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

XML

```xml
<?xml version="1.0" encoding="utf-8"?>
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/launcher_background" />
    <foreground android:drawable="@mipmap/ic_launcher_foreground" />
</adaptive-icon>
```